

FIT – HPUni BECOME A BETTER DEVELOPER

=====

BỘ BÀI TẬP THỰC HÀNH

MẢNG MỘT CHIỀU VÀ MẢNG HAI CHIỀU TRONG C++

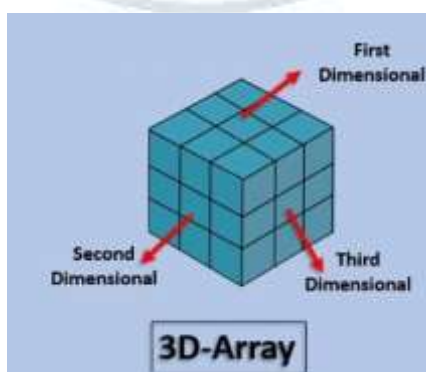
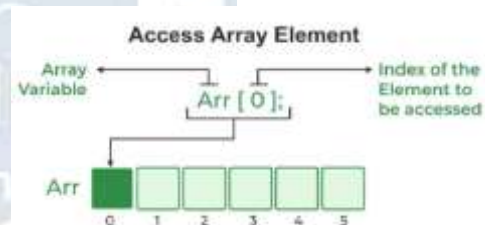
30 bài thực hành có Input - Output - Ví dụ - Thuật toán - Code minh họa

20 bài mảng một chiều | 10 bài ma trận | 15 bài có sử dụng hàm

Ngôn ngữ: C++11

	Column 0	Column 1	Column 2
Row 0	X[0][0]	X[0][1]	X[0][2]
Row 1	X[1][0]	X[1][1]	X[1][2]
Row 2	X[2][0]	X[2][1]	X[2][2]

2D-Array



HƯỚNG DẪN SỬ DỤNG TÀI LIỆU

Tài liệu được xây dựng theo hướng thực hành từ cơ bản đến vận dụng. Mỗi bài có 6 phần cố định: Nêu bài toán, Input, Output, Ví dụ, Thuật toán giải và Code minh họa.

Ký hiệu ★ đặt sau tên bài cho biết bài đó có sử dụng hàm tự định nghĩa. Tổng cộng có đúng 15 bài dùng hàm: Bài 1-10 và Bài 21-25.

Các chương trình minh họa sử dụng mảng kích thước cố định và cú pháp tương thích C++11. Trong lập trình C++ chuyên nghiệp, Bjarne Stroustrup lưu ý mảng built-in là một cơ chế mức thấp và thường nên được bao bọc bởi các cấu trúc mức cao hơn như `std::array` hoặc `std::vector`. Tuy nhiên, mảng built-in vẫn phù hợp cho mục tiêu thực hành chỉ số, vòng lặp và truyền dữ liệu vào hàm trong học phần nhập môn.

Tài liệu tham khảo chính: Bjarne Stroustrup, The C++ Programming Language, Fourth Edition, đặc biệt Chương 7 (Pointers, Arrays, and References) và Chương 12 (Functions).

Quy ước chung

- Chỉ số trong code C++ bắt đầu từ 0; khi đề bài yêu cầu “vị trí” cho người dùng, kết quả thường được trình bày từ 1.
- Các giới hạn n , m , p trong đề bài bảo đảm không vượt kích thước mảng tĩnh đã khai báo.
- Đầu vào được giả sử hợp lệ theo đúng ràng buộc của từng bài để tập trung vào kỹ năng xử lý mảng.
- Code được kiểm tra biên dịch với tùy chọn `-std=c++11`.

Danh mục bài tập

Số bài	Nội dung	Dùng hàm
1	Tính tổng các phần tử của mảng	Có
2	Đếm số phần tử chẵn và lẻ	Có
3	Tìm phần tử lớn nhất và nhỏ nhất	Có
4	Tính trung bình cộng và đếm phần tử lớn hơn trung bình	Có
5	Đảo ngược mảng	Có
6	Tìm vị trí xuất hiện đầu tiên của x	Có
7	Đếm số lần xuất hiện của x	Có
8	Sắp xếp mảng tăng dần bằng Selection Sort	Có
9	Tìm số lớn thứ hai phân biệt	Có
10	Loại bỏ phần tử trùng lặp, giữ thứ tự xuất hiện	Có
11	Chèn phần tử x vào vị trí k	Không
12	Xóa phần tử tại vị trí k	Không
13	Xoay mảng sang phải k vị trí	Không
14	Kiểm tra mảng đối xứng	Không
15	Đếm số dương, số âm và số 0	Không
16	Tìm đoạn tăng liên tiếp dài nhất	Không
17	Trộn hai mảng đã tăng thành một mảng tăng	Không
18	Thống kê tần suất các giá trị	Không
19	Tìm giao của hai mảng, không lặp	Không
20	Đưa tất cả số 0 về cuối mảng	Không
21	Tính tổng các phần tử của ma trận	Có
22	Tính tổng từng hàng và tìm hàng có tổng lớn nhất	Có
23	Tính tổng từng cột và tìm cột có tổng nhỏ nhất	Có
24	Tính tổng đường chéo chính và đường chéo phụ	Có
25	Chuyển vị ma trận	Có
26	Tìm giá trị lớn nhất và tất cả vị trí xuất hiện	Không
27	Đếm số nguyên tố trong ma trận	Không
28	Tìm hàng có tổng lớn nhất không dùng hàm	Không
29	Nhân hai ma trận	Không
30	Kiểm tra ma trận đối xứng	Không

CHÚ Ý KHI THỰC HÀNH:

Bộ bài tập được xây dựng thành ba mức: Mức 1 (Bài 1-7, 11-15, 21, 24, 26-27), Mức 2 (Bài 8-10, 16-20, 22-23, 25, 28, 30), Mức 3 (Bài 29 và các biến thể mở rộng).

Biến thể mở rộng cho sinh viên thực hành:

- Thay kiểu `int` bằng `double` ở các bài phù hợp và phân tích ảnh hưởng đến phép so sánh.
- Bổ sung kiểm tra dữ liệu đầu vào và xử lý trường hợp vượt giới hạn kích thước mảng.
- Viết lại các bài chưa dùng hàm theo hướng mô-đun hóa; so sánh độ dễ đọc với bản chỉ dùng `main()`.
- Viết lại bằng `std::array` hoặc `std::vector` để đối chiếu với mảng built-in.
- Phân tích độ phức tạp thời gian và bộ nhớ của từng thuật toán.



PHẦN I. BÀI TẬP MẢNG MỘT CHIỀU

Mục tiêu: luyện nhập/xuất mảng, duyệt mảng, tìm kiếm, thống kê, biến đổi, sắp xếp và các kỹ thuật hai chỉ số/copy mảng cơ bản.

Bài 1. Tính tổng các phần tử của mảng ★

1. Nêu bài toán

Cho mảng một chiều gồm n số nguyên. Hãy tính tổng tất cả các phần tử trong mảng.

2. Input

Dòng 1 chứa số nguyên n ($1 \leq n \leq 1000$). Dòng 2 chứa n số nguyên $a[i]$.

3. Output

In ra tổng các phần tử của mảng.

4. Ví dụ

INPUT	OUTPUT
5 1 2 3 4 5	15

5. Thuật toán giải

Bước 1: Đọc n và n phần tử vào mảng a .

Bước 2: Duyệt i từ 0 đến $n-1$, cộng $a[i]$ vào biến tổng.

Bước 3: In giá trị tổng. Hàm `sumArray()` thực hiện phép cộng để tách phần xử lý khỏi `main()`.

6. Code minh họa

```
#include <iostream>
using namespace std;
long long sumArray(const int a[], int n) {
    long long sum = 0;
    for (int i = 0; i < n; ++i) sum += a[i];
    return sum;
}
int main() {
    const int MAXN = 1000;
    int n, a[MAXN];
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];
    cout << sumArray(a, n);
    return 0;
}
```

Bài 2. Đếm số phần tử chẵn và lẻ ★

1. Nêu bài toán

Cho mảng n số nguyên. Hãy đếm có bao nhiêu phần tử chẵn và bao nhiêu phần tử lẻ.

2. Input

Dòng 1: n ($1 \leq n \leq 1000$). Dòng 2: n số nguyên.

3. Output

In lần lượt số lượng phần tử chẵn và số lượng phần tử lẻ.

4. Ví dụ

INPUT	OUTPUT
6 1 2 3 4 5 6	Chan: 3 Le: 3

5. Thuật toán giải

Bước 1: Khởi tạo hai biến đếm evenCount và oddCount bằng 0.

Bước 2: Duyệt từng phần tử; nếu chia hết cho 2 thì tăng evenCount, ngược lại tăng oddCount.

Bước 3: Trả kết quả qua hai tham chiếu trong hàm countEvenOdd().

6. Code minh họa

```
#include <iostream>
using namespace std;

void countEvenOdd(const int a[], int n, int& evenCount, int& oddCount) {
    evenCount = oddCount = 0;
    for (int i = 0; i < n; ++i) {
        if (a[i] % 2 == 0) ++evenCount;
        else ++oddCount;
    }
}

int main() {
    const int MAXN = 1000;
    int n, a[MAXN], evenCount, oddCount;
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];
    countEvenOdd(a, n, evenCount, oddCount);
    cout << "Chan: " << evenCount << '\n';
    cout << "Le: " << oddCount;
    return 0;
}
```

Bài 3. Tìm phần tử lớn nhất và nhỏ nhất ★

1. Nêu bài toán

Cho mảng n số nguyên. Hãy xác định giá trị lớn nhất và giá trị nhỏ nhất trong mảng.

2. Input

Dòng 1: n ($1 \leq n \leq 1000$). Dòng 2: n số nguyên.

3. Output

In ra giá trị lớn nhất và nhỏ nhất.

4. Ví dụ

INPUT	OUTPUT
6 -2 7 4 7 0 3	Max: 7 Min: -2

5. Thuật toán giải

Bước 1: Gán maxVal và minVal bằng phần tử đầu tiên.

Bước 2: Duyệt từ phần tử thứ hai: cập nhật maxVal nếu gặp giá trị lớn hơn, cập nhật minVal nếu gặp giá trị nhỏ hơn.

Bước 3: Hàm findMinMax() trả hai kết quả bằng tham chiếu.

6. Code minh họa

```
#include <iostream>
using namespace std;

void findMinMax(const int a[], int n, int& minVal, int& maxVal) {
    minVal = maxVal = a[0];
    for (int i = 1; i < n; ++i) {
        if (a[i] < minVal) minVal = a[i];
        if (a[i] > maxVal) maxVal = a[i];
    }
}

int main() {
    const int MAXN = 1000;
```

```

int n, a[MAXN], minVal, maxVal;
cin >> n;
for (int i = 0; i < n; ++i) cin >> a[i];
findMinMax(a, n, minVal, maxVal);
cout << "Max: " << maxVal << '\n';
cout << "Min: " << minVal;
return 0;
}

```

Bài 4. Tính trung bình cộng và đếm phần tử lớn hơn trung bình ★

1. Nêu bài toán

Cho mảng n số nguyên. Tính trung bình cộng của mảng và đếm số phần tử có giá trị lớn hơn trung bình cộng đó.

2. Input

Dòng 1: n ($1 \leq n \leq 1000$). Dòng 2: n số nguyên.

3. Output

Dòng 1 in trung bình cộng với 2 chữ số thập phân. Dòng 2 in số phần tử lớn hơn trung bình.

4. Ví dụ

INPUT	OUTPUT
5 1 2 3 4 10	Trung bình: 4.00 Số phần tử lớn hơn TB: 1

5. Thuật toán giải

Bước 1: Dùng hàm `average()` tính tổng và chia cho n theo kiểu `double`.

Bước 2: Dùng hàm `countGreaterThan()` duyệt mảng và đếm $a[i] > \text{avg}$.

Bước 3: In trung bình với `fixed` và `setprecision(2)`, sau đó in số lượng.

6. Code minh họa

```

#include <iostream>
#include <iomanip>
using namespace std;

double average(const int a[], int n) {
    long long sum = 0;
    for (int i = 0; i < n; ++i) sum += a[i];
    return static_cast<double>(sum) / n;
}

int countGreaterThan(const int a[], int n, double value) {
    int count = 0;
    for (int i = 0; i < n; ++i)
        if (a[i] > value) ++count;
    return count;
}

int main() {
    const int MAXN = 1000;
    int n, a[MAXN];
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];
    double avg = average(a, n);
    cout << fixed << setprecision(2);
    cout << "Trung bình: " << avg << '\n';
    cout << "Số phần tử lớn hơn TB: "
         << countGreaterThan(a, n, avg);
    return 0;
}

```

Bài 5. Đảo ngược mảng ★

1. Nêu bài toán

Cho mảng n số nguyên. Hãy đảo ngược thứ tự các phần tử ngay trên mảng ban đầu.

2. Input

Dòng 1: n ($1 \leq n \leq 1000$). Dòng 2: n số nguyên.

3. Output

In mảng sau khi đảo ngược.

4. Ví dụ

INPUT	OUTPUT
5 1 2 3 4 5	5 4 3 2 1

5. Thuật toán giải

Bước 1: Đặt hai chỉ số $l = 0$ và $r = n - 1$.

Bước 2: Trong khi $l < r$, hoán đổi $a[l]$ và $a[r]$, sau đó tăng l và giảm r .

Bước 3: Hàm `reverseArray()` biến đổi trực tiếp mảng; `main()` chỉ nhập và xuất.

6. Code minh họa

```
#include <iostream>
#include <utility>
using namespace std;

void reverseArray(int a[], int n) {
    int l = 0, r = n - 1;
    while (l < r) {
        swap(a[l], a[r]);
        ++l;
        --r;
    }
}

int main() {
    const int MAXN = 1000;
    int n, a[MAXN];
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];
    reverseArray(a, n);
    for (int i = 0; i < n; ++i) {
        if (i) cout << ' ';
        cout << a[i];
    }
    return 0;
}
```

Bài 6. Tìm vị trí xuất hiện đầu tiên của x ★

1. Nêu bài toán

Cho mảng n số nguyên và số nguyên x . Hãy tìm vị trí xuất hiện đầu tiên của x trong mảng. Vị trí được đánh số từ 1.

2. Input

Dòng 1: n . Dòng 2: n số nguyên. Dòng 3: x .

3. Output

In vị trí đầu tiên của x ; nếu không tìm thấy, in -1.

4. Ví dụ

INPUT	OUTPUT
-------	--------


```
7
4 2 9 2 5 1 2
2
```

2

5. Thuật toán giải

Bước 1: Duyệt mảng từ trái sang phải.

Bước 2: Ngay khi $a[i] == x$, trả về $i + 1$ để chuyển sang chỉ số 1-based.

Bước 3: Nếu duyệt hết mà không thấy, hàm `findFirst()` trả về -1.

6. Code minh họa

```
#include <iostream>
using namespace std;

int findFirst(const int a[], int n, int x) {
    for (int i = 0; i < n; ++i)
        if (a[i] == x) return i + 1;
    return -1;
}

int main() {
    const int MAXN = 1000;
    int n, a[MAXN], x;
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];
    cin >> x;
    cout << findFirst(a, n, x);
    return 0;
}
```

Bài 7. Đếm số lần xuất hiện của x ★

1. Nêu bài toán

Cho mảng n số nguyên và một số nguyên x . Hãy đếm số lần x xuất hiện trong mảng.

2. Input

Dòng 1: n . Dòng 2: n số nguyên. Dòng 3: x .

3. Output

In số lần xuất hiện của x .

4. Ví dụ

INPUT	OUTPUT
<pre>7 4 2 9 2 5 1 2 2</pre>	<pre>3</pre>

5. Thuật toán giải

Bước 1: Khởi tạo `count = 0`.

Bước 2: Duyệt toàn bộ mảng; khi $a[i] == x$ thì tăng `count`.

Bước 3: Hàm `countOccurrences()` trả về `count`.

6. Code minh họa

```
#include <iostream>
using namespace std;

int countOccurrences(const int a[], int n, int x) {
    int count = 0;
    for (int i = 0; i < n; ++i)
        if (a[i] == x) ++count;
    return count;
}

int main() {
```



```

const int MAXN = 1000;
int n, a[MAXN], x;
cin >> n;
for (int i = 0; i < n; ++i) cin >> a[i];
cin >> x;
cout << countOccurrences(a, n, x);
return 0;
}

```

Bài 8. Sắp xếp mảng tăng dần bằng Selection Sort ★

1. Nêu bài toán

Cho mảng n số nguyên. Hãy sắp xếp mảng theo thứ tự tăng dần bằng thuật toán chọn trực tiếp (Selection Sort).

2. Input

Dòng 1: n ($1 \leq n \leq 1000$). Dòng 2: n số nguyên.

3. Output

In mảng sau khi sắp xếp tăng dần.

4. Ví dụ

INPUT	OUTPUT
6 5 1 4 2 8 0	0 1 2 4 5 8

5. Thuật toán giải

Bước 1: Với mỗi vị trí i từ 0 đến $n-2$, giả sử i là vị trí nhỏ nhất.

Bước 2: Tìm phần tử nhỏ nhất trong đoạn $i+1..n-1$.

Bước 3: Hoán đổi phần tử nhỏ nhất tìm được với $a[i]$. Hàm `selectionSort()` thực hiện toàn bộ quá trình.

6. Code minh họa

```

#include <iostream>
#include <utility>
using namespace std;

void selectionSort(int a[], int n) {
    for (int i = 0; i < n - 1; ++i) {
        int minPos = i;
        for (int j = i + 1; j < n; ++j)
            if (a[j] < a[minPos]) minPos = j;
        if (minPos != i) swap(a[i], a[minPos]);
    }
}

int main() {
    const int MAXN = 1000;
    int n, a[MAXN];
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];
    selectionSort(a, n);
    for (int i = 0; i < n; ++i) {
        if (i) cout << ' ';
        cout << a[i];
    }
    return 0;
}

```

Bài 9. Tìm số lớn thứ hai phân biệt ★

1. Nêu bài toán

Cho mảng n số nguyên. Hãy tìm giá trị lớn thứ hai khác với giá trị lớn nhất. Nếu không tồn tại thì thông báo KHÔNG CÓ.

2. Input

Dòng 1: n ($2 \leq n \leq 1000$). Dòng 2: n số nguyên.

3. Output

In giá trị lớn thứ hai phân biệt hoặc chuỗi KHONG CO.

4. Ví dụ

INPUT	OUTPUT
7 5 1 9 9 7 3 7	7

5. Thuật toán giải

Bước 1: Tìm giá trị lớn nhất maxVal.

Bước 2: Duyệt lại mảng và tìm giá trị lớn nhất nhưng nhỏ hơn maxVal.

Bước 3: Hàm secondLargest() trả true nếu tìm được và ghi kết quả vào biến second; ngược lại trả false.

6. Code minh họa

```
#include <iostream>
using namespace std;

bool secondLargest(const int a[], int n, int& second) {
    int maxVal = a[0];
    for (int i = 1; i < n; ++i)
        if (a[i] > maxVal) maxVal = a[i];

    bool found = false;
    for (int i = 0; i < n; ++i) {
        if (a[i] < maxVal && (!found || a[i] > second)) {
            second = a[i];
            found = true;
        }
    }
    return found;
}

int main() {
    const int MAXN = 1000;
    int n, a[MAXN], second;
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];
    if (secondLargest(a, n, second)) cout << second;
    else cout << "KHONG CO";
    return 0;
}
```

Bài 10. Loại bỏ phần tử trùng lặp, giữ thứ tự xuất hiện ★**1. Nêu bài toán**

Cho mảng n số nguyên. Hãy loại bỏ các phần tử trùng lặp sao cho mỗi giá trị chỉ còn lần xuất hiện đầu tiên và thứ tự tương đối không thay đổi.

2. Input

Dòng 1: n ($1 \leq n \leq 1000$). Dòng 2: n số nguyên.

3. Output

Dòng 1 in số phần tử còn lại. Dòng 2 in mảng sau khi loại trùng.

4. Ví dụ

INPUT	OUTPUT
8 1 2 1 3 2 4 4 5	5 1 2 3 4 5

5. Thuật toán giải**Bước 1:** Duyệt từng phần tử $a[i]$.**Bước 2:** Kiểm tra $a[i]$ đã xuất hiện trong đoạn kết quả $a[0..m-1]$ hay chưa.**Bước 3:** Nếu chưa có, gán $a[m] = a[i]$ và tăng m . Hàm `uniqueInPlace()` trả về kích thước mới m .**6. Code minh họa**

```
#include <iostream>
using namespace std;

int uniqueInPlace(int a[], int n) {
    int m = 0;
    for (int i = 0; i < n; ++i) {
        bool existed = false;
        for (int j = 0; j < m; ++j)
            if (a[j] == a[i]) {
                existed = true;
                break;
            }
        if (!existed) a[m++] = a[i];
    }
    return m;
}

int main() {
    const int MAXN = 1000;
    int n, a[MAXN];
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];
    n = uniqueInPlace(a, n);
    cout << n << '\n';
    for (int i = 0; i < n; ++i) {
        if (i) cout << ' ';
        cout << a[i];
    }
    return 0;
}
```

Bài 11. Chèn phần tử x vào vị trí k**1. Nêu bài toán**

Cho mảng n số nguyên, số nguyên x và vị trí k (đánh số từ 1). Hãy chèn x vào vị trí k , các phần tử từ k trở đi dịch sang phải một vị trí.

2. Input

Dòng 1: n ($1 \leq n < 1000$). Dòng 2: n số nguyên. Dòng 3: x và k , với $1 \leq k \leq n+1$.

3. Output

In mảng sau khi chèn.

4. Ví dụ

INPUT	OUTPUT
5 1 2 3 4 5 9 3	1 2 9 3 4 5

5. Thuật toán giải**Bước 1:** Chuyển vị trí k từ 1-based sang chỉ số $k-1$.**Bước 2:** Dịch các phần tử từ cuối mảng về vị trí k sang phải một ô, duyệt từ n xuống k .**Bước 3:** Gán $a[k-1] = x$, tăng n và in mảng mới.**6. Code minh họa**

```
#include <iostream>
using namespace std;
```

```

int main() {
    const int MAXN = 1000;
    int n, a[MAXN], x, k;
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];
    cin >> x >> k;

    for (int i = n; i >= k; --i) a[i] = a[i - 1];
    a[k - 1] = x;
    ++n;

    for (int i = 0; i < n; ++i) {
        if (i) cout << ' ';
        cout << a[i];
    }
    return 0;
}

```

Bài 12. Xóa phần tử tại vị trí k

1. Nêu bài toán

Cho mảng n số nguyên và vị trí k (đánh số từ 1). Hãy xóa phần tử tại vị trí k.

2. Input

Dòng 1: n ($1 \leq n \leq 1000$). Dòng 2: n số nguyên. Dòng 3: k, với $1 \leq k \leq n$.

3. Output

In mảng sau khi xóa.

4. Ví dụ

INPUT	OUTPUT
6 10 20 30 40 50 60 4	10 20 30 50 60

5. Thuật toán giải

Bước 1: Phần tử cần xóa có chỉ số k-1.

Bước 2: Dịch các phần tử từ chỉ số k đến n-1 sang trái một vị trí.

Bước 3: Giảm n đi 1 rồi in n phần tử đầu.

6. Code minh họa

```

#include <iostream>
using namespace std;

int main() {
    const int MAXN = 1000;
    int n, a[MAXN], k;
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];
    cin >> k;

    for (int i = k - 1; i < n - 1; ++i)
        a[i] = a[i + 1];
    --n;

    for (int i = 0; i < n; ++i) {
        if (i) cout << ' ';
        cout << a[i];
    }
    return 0;
}

```

Bài 13. Xoay mảng sang phải k vị trí

1. Nêu bài toán

Cho mảng n số nguyên và số nguyên không âm k. Hãy xoay toàn bộ mảng sang phải k vị trí.

2. Input

Dòng 1: n ($1 \leq n \leq 1000$). Dòng 2: n số nguyên. Dòng 3: k ($k \geq 0$).

3. Output

In mảng sau khi xoay phải k vị trí.

4. Ví dụ

INPUT	OUTPUT
6 1 2 3 4 5 6 2	5 6 1 2 3 4

5. Thuật toán giải

Bước 1: Rút gọn k bằng $k \% n$.

Bước 2: Với mỗi vị trí i, phần tử $a[i]$ sẽ chuyển đến vị trí $(i+k) \% n$ trong mảng tạm.

Bước 3: Sao chép mảng tạm trở lại a và in kết quả.

6. Code minh họa

```
#include <iostream>
using namespace std;

int main() {
    const int MAXN = 1000;
    int n, a[MAXN], temp[MAXN];
    long long k;
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];
    cin >> k;

    int shift = static_cast<int>(k % n);
    for (int i = 0; i < n; ++i)
        temp[(i + shift) % n] = a[i];
    for (int i = 0; i < n; ++i) a[i] = temp[i];

    for (int i = 0; i < n; ++i) {
        if (i) cout << ' ';
        cout << a[i];
    }
    return 0;
}
```

Bài 14. Kiểm tra mảng đối xứng

1. Nêu bài toán

Một mảng được gọi là đối xứng nếu đọc từ trái sang phải hay từ phải sang trái đều cho cùng dãy giá trị. Hãy kiểm tra mảng đã cho có đối xứng hay không.

2. Input

Dòng 1: n ($1 \leq n \leq 1000$). Dòng 2: n số nguyên.

3. Output

In YES nếu mảng đối xứng, ngược lại in NO.

4. Ví dụ

INPUT	OUTPUT
5 1 2 3 2 1	YES

5. Thuật toán giải

Bước 1: So sánh từng cặp $a[i]$ và $a[n-1-i]$ với i từ 0 đến $n/2-1$.

Bước 2: Nếu có một cặp khác nhau thì mảng không đối xứng.

Bước 3: Nếu tất cả các cặp đều bằng nhau thì in YES.

6. Code minh họa

```
#include <iostream>
using namespace std;

int main() {
    const int MAXN = 1000;
    int n, a[MAXN];
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];

    bool symmetric = true;
    for (int i = 0; i < n / 2; ++i) {
        if (a[i] != a[n - 1 - i]) {
            symmetric = false;
            break;
        }
    }
    cout << (symmetric ? "YES" : "NO");
    return 0;
}
```

Bài 15. Đếm số dương, số âm và số 0**1. Nêu bài toán**

Cho mảng n số nguyên. Hãy đếm số phần tử dương, số phần tử âm và số phần tử bằng 0.

2. Input

Dòng 1: n . Dòng 2: n số nguyên.

3. Output

In ba dòng lần lượt là số lượng phần tử dương, âm và bằng 0.

4. Ví dụ

INPUT	OUTPUT
7 -1 0 2 -3 0 5 7	Duong: 3 Am: 2 Khong: 2

5. Thuật toán giải

Bước 1: Khởi tạo ba biến đếm pos, neg, zero bằng 0.

Bước 2: Duyệt từng phần tử và phân loại theo $a[i] > 0$, $a[i] < 0$ hoặc $a[i] == 0$.

Bước 3: In ba biến đếm.

6. Code minh họa

```
#include <iostream>
using namespace std;

int main() {
    const int MAXN = 1000;
    int n, a[MAXN], pos = 0, neg = 0, zero = 0;
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];

    for (int i = 0; i < n; ++i) {
        if (a[i] > 0) ++pos;
        else if (a[i] < 0) ++neg;
        else ++zero;
    }
}
```

```

cout << "Duong: " << pos << '\n';
cout << "Am: " << neg << '\n';
cout << "Khong: " << zero;
return 0;
}

```

Bài 16. Tìm đoạn tăng liên tiếp dài nhất

1. Nêu bài toán

Cho mảng n số nguyên. Hãy tìm đoạn liên tiếp dài nhất mà các phần tử tăng nghiêm ngặt, tức phần tử sau lớn hơn phần tử trước.

2. Input

Dòng 1: n ($1 \leq n \leq 1000$). Dòng 2: n số nguyên.

3. Output

Dòng 1 in độ dài đoạn tăng dài nhất. Dòng 2 in các phần tử của đoạn đó. Nếu có nhiều đoạn cùng độ dài, lấy đoạn xuất hiện trước.

4. Ví dụ

INPUT	OUTPUT
8 1 2 5 3 4 6 7 2	Do dai: 4 Doan: 3 4 6 7

5. Thuật toán giải

Bước 1: Duy trì độ dài đoạn hiện tại $curLen$ và vị trí bắt đầu $curStart$.

Bước 2: Nếu $a[i] > a[i-1]$, tăng $curLen$; ngược lại bắt đầu đoạn mới tại i .

Bước 3: Mỗi khi $curLen > bestLen$, cập nhật $bestLen$ và $bestStart$. Cuối cùng in đoạn $a[bestStart..bestStart+bestLen-1]$.

6. Code minh họa

```

#include <iostream>
using namespace std;

int main() {
    const int MAXN = 1000;
    int n, a[MAXN];
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];

    int curStart = 0, curLen = 1;
    int bestStart = 0, bestLen = 1;
    for (int i = 1; i < n; ++i) {
        if (a[i] > a[i - 1]) {
            ++curLen;
        } else {
            curStart = i;
            curLen = 1;
        }
        if (curLen > bestLen) {
            bestLen = curLen;
            bestStart = curStart;
        }
    }

    cout << "Do dai: " << bestLen << '\n';
    cout << "Doan: ";
    for (int i = bestStart; i < bestStart + bestLen; ++i) {
        if (i > bestStart) cout << ' ';
        cout << a[i];
    }
    return 0;
}

```


Bài 17. Trộn hai mảng đã tăng thành một mảng tăng

1. Nêu bài toán

Cho hai mảng A và B đã được sắp xếp tăng dần. Hãy trộn chúng thành mảng C cũng tăng dần mà không thực hiện sắp xếp lại toàn bộ.

2. Input

Dòng 1: n, dòng 2: n phần tử A. Dòng 3: m, dòng 4: m phần tử B. Giả sử $n+m \leq 2000$.

3. Output

In mảng C sau khi trộn.

4. Ví dụ

INPUT	OUTPUT
4 1 3 5 7 3 2 3 6	1 2 3 3 5 6 7

5. Thuật toán giải

Bước 1: Dùng hai chỉ số i, j lần lượt trở vào đầu A và B.

Bước 2: So sánh $A[i]$ với $B[j]$, đưa phần tử nhỏ hơn vào C rồi tăng chỉ số tương ứng.

Bước 3: Khi một mảng hết phần tử, chép toàn bộ phần còn lại của mảng kia vào C.

6. Code minh họa

```
#include <iostream>
using namespace std;

int main() {
    const int MAXN = 1000;
    int n, m, a[MAXN], b[MAXN], c[2 * MAXN];
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];
    cin >> m;
    for (int i = 0; i < m; ++i) cin >> b[i];

    int i = 0, j = 0, k = 0;
    while (i < n && j < m) {
        if (a[i] <= b[j]) c[k++] = a[i++];
        else c[k++] = b[j++];
    }
    while (i < n) c[k++] = a[i++];
    while (j < m) c[k++] = b[j++];

    for (int t = 0; t < k; ++t) {
        if (t) cout << ' ';
        cout << c[t];
    }
    return 0;
}
```

Bài 18. Thống kê tần suất các giá trị

1. Nêu bài toán

Cho mảng n số nguyên. Với mỗi giá trị phân biệt, hãy in số lần xuất hiện của nó theo thứ tự xuất hiện đầu tiên trong mảng.

2. Input

Dòng 1: n ($1 \leq n \leq 1000$). Dòng 2: n số nguyên.

3. Output

Mỗi dòng gồm một giá trị và số lần xuất hiện của giá trị đó.

4. Ví dụ

INPUT	OUTPUT
8	1 3
1 2 1 3 2 1 4 3	2 2
	3 2
	4 1

5. Thuật toán giải

Bước 1: Dùng mảng counted để đánh dấu vị trí đã được tính tần suất.

Bước 2: Với mỗi i chưa được đánh dấu, đặt count = 1 và duyệt j > i; nếu a[j] == a[i] thì tăng count và đánh dấu counted[j].

Bước 3: In a[i] và count. Độ phức tạp $O(n^2)$, phù hợp bài luyện mảng cơ bản.

6. Code minh họa

```
#include <iostream>
using namespace std;

int main() {
    const int MAXN = 1000;
    int n, a[MAXN];
    bool counted[MAXN] = {false};
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];

    for (int i = 0; i < n; ++i) {
        if (counted[i]) continue;
        int count = 1;
        for (int j = i + 1; j < n; ++j) {
            if (a[j] == a[i]) {
                ++count;
                counted[j] = true;
            }
        }
        cout << a[i] << ' ' << count << '\n';
    }
    return 0;
}
```

Bài 19. Tìm giao của hai mảng, không lặp

1. Nêu bài toán

Cho hai mảng số nguyên A và B. Hãy in các giá trị xuất hiện trong cả hai mảng, mỗi giá trị chỉ in một lần và theo thứ tự xuất hiện trong A.

2. Input

Dòng 1: n, dòng 2: n phần tử A. Dòng 3: m, dòng 4: m phần tử B. Mỗi kích thước không quá 1000.

3. Output

In các phần tử thuộc giao. Nếu không có phần tử chung, in RONG.

4. Ví dụ

INPUT	OUTPUT
6	2 4
1 2 2 3 4 5	
4	
2 4 4 6	

5. Thuật toán giải

Bước 1: Duyệt từng giá trị A[i].

Bước 2: Kiểm tra A[i] có xuất hiện trong B và chưa từng được in trước đó hay không.

Bước 3: Nếu thỏa cả hai điều kiện, in A[i]. Dùng kiểm tra tuyến tính để giữ bài ở mức mảng cơ bản.

6. Code minh họa

```
#include <iostream>
using namespace std;

int main() {
    const int MAXN = 1000;
    int n, m, a[MAXN], b[MAXN], out[MAXN], outSize = 0;
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];
    cin >> m;
    for (int i = 0; i < m; ++i) cin >> b[i];

    for (int i = 0; i < n; ++i) {
        bool inB = false, printed = false;
        for (int j = 0; j < m; ++j)
            if (a[i] == b[j]) { inB = true; break; }
        for (int j = 0; j < outSize; ++j)
            if (a[i] == out[j]) { printed = true; break; }
        if (inB && !printed) out[outSize++] = a[i];
    }

    if (outSize == 0) cout << "RONG";
    else {
        for (int i = 0; i < outSize; ++i) {
            if (i) cout << ' ';
            cout << out[i];
        }
    }
    return 0;
}
```

Bài 20. Đưa tất cả số 0 về cuối mảng

1. Nêu bài toán

Cho mảng n số nguyên. Hãy đưa tất cả phần tử bằng 0 về cuối mảng nhưng vẫn giữ nguyên thứ tự tương đối của các phần tử khác 0.

2. Input

Dòng 1: n ($1 \leq n \leq 1000$). Dòng 2: n số nguyên.

3. Output

In mảng sau khi di chuyển các số 0.

4. Ví dụ

INPUT	OUTPUT
8 0 1 0 3 12 0 5 0	1 3 12 5 0 0 0 0

5. Thuật toán giải

Bước 1: Dùng chỉ số $write = 0$ để ghi lần lượt các phần tử khác 0 về đầu mảng.

Bước 2: Sau khi duyệt xong, các vị trí từ $write$ đến $n-1$ được gán bằng 0.

Bước 3: Cách làm chỉ cần $O(1)$ bộ nhớ phụ và giữ nguyên thứ tự của phần tử khác 0.

6. Code minh họa

```
#include <iostream>
using namespace std;

int main() {
    const int MAXN = 1000;
    int n, a[MAXN];
    cin >> n;
    for (int i = 0; i < n; ++i) cin >> a[i];

    int write = 0;
    for (int i = 0; i < n; ++i)
```

```

    if (a[i] != 0) a[write++] = a[i];
    while (write < n) a[write++] = 0;

    for (int i = 0; i < n; ++i) {
        if (i) cout << ' ';
        cout << a[i];
    }
    return 0;
}

```

PHẦN II. BÀI TẬP MẢNG HAI CHIỀU (MA TRẬN)

Mục tiêu: luyện duyệt theo hàng/cột, đường chéo, chuyển vị, thống kê trên ma trận và phép nhân ma trận.

Bài 21. Tính tổng các phần tử của ma trận ★

1. Nêu bài toán

Cho ma trận số nguyên có m hàng và n cột. Hãy tính tổng tất cả các phần tử trong ma trận.

2. Input

Dòng 1: m và n ($1 \leq m, n \leq 100$). Tiếp theo là m dòng, mỗi dòng chứa n số nguyên.

3. Output

In tổng tất cả phần tử của ma trận.

4. Ví dụ

INPUT	OUTPUT
2 3 1 2 3 4 5 6	21

5. Thuật toán giải

Bước 1: Duyệt từng hàng i và từng cột j.

Bước 2: Cộng $a[i][j]$ vào biến tổng.

Bước 3: Hàm sumMatrix() nhận ma trận và kích thước, sau đó trả về tổng.

6. Code minh họa

```

#include <iostream>
using namespace std;

const int MAX = 100;

long long sumMatrix(const int a[][MAX], int m, int n) {
    long long sum = 0;
    for (int i = 0; i < m; ++i)
        for (int j = 0; j < n; ++j)
            sum += a[i][j];
    return sum;
}

int main() {
    int m, n, a[MAX][MAX];
    cin >> m >> n;
    for (int i = 0; i < m; ++i)
        for (int j = 0; j < n; ++j) cin >> a[i][j];
    cout << sumMatrix(a, m, n);
    return 0;
}

```

Bài 22. Tính tổng từng hàng và tìm hàng có tổng lớn nhất ★

1. Nêu bài toán

Cho ma trận $m \times n$ số nguyên. Hãy tính tổng của từng hàng và xác định hàng có tổng lớn nhất. Nếu nhiều hàng cùng tổng lớn nhất, chọn hàng có chỉ số nhỏ nhất.

2. Input

Dòng 1: m, n . Tiếp theo là m dòng, mỗi dòng n số nguyên.

3. Output

In tổng từng hàng. Dòng cuối in số thứ tự hàng có tổng lớn nhất (đánh số từ 1).

4. Ví dụ

INPUT	OUTPUT
3 3	Hàng 1: 6
1 2 3	Hàng 2: 15
4 5 6	Hàng 3: 0
-1 0 1	Hàng lớn nhất: 2

5. Thuật toán giải

Bước 1: Hàm `rowSum()` tính tổng một hàng r bằng cách duyệt các cột.

Bước 2: Trong `main()`, gọi `rowSum()` cho từng hàng và in tổng.

Bước 3: So sánh các tổng để lưu `bestRow`; chỉ cập nhật khi tổng mới lớn hơn để giữ hàng xuất hiện trước nếu hòa.

6. Code minh họa

```
#include <iostream>
using namespace std;

const int MAX = 100;

long long rowSum(const int a[][MAX], int row, int n) {
    long long sum = 0;
    for (int j = 0; j < n; ++j) sum += a[row][j];
    return sum;
}

int main() {
    int m, n, a[MAX][MAX];
    cin >> m >> n;
    for (int i = 0; i < m; ++i)
        for (int j = 0; j < n; ++j) cin >> a[i][j];

    int bestRow = 0;
    long long bestSum = rowSum(a, 0, n);
    for (int i = 0; i < m; ++i) {
        long long s = rowSum(a, i, n);
        cout << "Hàng " << i + 1 << ": " << s << '\n';
        if (s > bestSum) {
            bestSum = s;
            bestRow = i;
        }
    }
    cout << "Hàng lớn nhất: " << bestRow + 1;
    return 0;
}
```

Bài 23. Tính tổng từng cột và tìm cột có tổng nhỏ nhất ★

1. Nêu bài toán

Cho ma trận $m \times n$ số nguyên. Hãy tính tổng từng cột và tìm cột có tổng nhỏ nhất. Nếu hòa, chọn cột xuất hiện trước.

2. Input

Dòng 1: m, n. Tiếp theo là m dòng dữ liệu ma trận.

3. Output

In tổng từng cột và số thứ tự cột có tổng nhỏ nhất, đánh số từ 1.

4. Ví dụ

INPUT	OUTPUT
3 2	Cot 1: 6
1 4	Cot 2: 15
2 5	Cot nhỏ nhất: 1
3 6	

5. Thuật toán giải

Bước 1: Hàm colSum() cộng $a[i][col]$ với i từ 0 đến $m-1$.

Bước 2: Gọi colSum() cho từng cột và in kết quả.

Bước 3: Duy trì bestCol và bestSum; cập nhật khi gặp tổng nhỏ hơn.

6. Code minh họa

```
#include <iostream>
using namespace std;

const int MAX = 100;

long long colSum(const int a[][MAX], int m, int col) {
    long long sum = 0;
    for (int i = 0; i < m; ++i) sum += a[i][col];
    return sum;
}

int main() {
    int m, n, a[MAX][MAX];
    cin >> m >> n;
    for (int i = 0; i < m; ++i)
        for (int j = 0; j < n; ++j) cin >> a[i][j];

    int bestCol = 0;
    long long bestSum = colSum(a, m, 0);
    for (int j = 0; j < n; ++j) {
        long long s = colSum(a, m, j);
        cout << "Cot " << j + 1 << ": " << s << '\n';
        if (s < bestSum) {
            bestSum = s;
            bestCol = j;
        }
    }
    cout << "Cot nhỏ nhất: " << bestCol + 1;
    return 0;
}
```

Bài 24. Tính tổng đường chéo chính và đường chéo phụ ★**1. Nêu bài toán**

Cho ma trận vuông $n \times n$. Hãy tính tổng các phần tử trên đường chéo chính và tổng các phần tử trên đường chéo phụ.

2. Input

Dòng 1: n ($1 \leq n \leq 100$). Tiếp theo là n dòng, mỗi dòng n số nguyên.

3. Output

In tổng đường chéo chính và tổng đường chéo phụ.

4. Ví dụ

INPUT	OUTPUT
-------	--------

3
1 2 3
4 5 6
7 8 9

Cheo chính: 15
Cheo phụ: 15

5. Thuật toán giải

Bước 1: Đường chéo chính gồm các phần tử $a[i][i]$.

Bước 2: Đường chéo phụ gồm các phần tử $a[i][n-1-i]$.

Bước 3: Hàm `diagonalSums()` duyệt i từ 0 đến $n-1$ và cộng đồng thời hai tổng, trả qua tham chiếu.

6. Code minh họa

```
#include <iostream>
using namespace std;

const int MAX = 100;

void diagonalSums(const int a[][MAX], int n,
                  long long& mainDiag, long long& secondDiag) {
    mainDiag = secondDiag = 0;
    for (int i = 0; i < n; ++i) {
        mainDiag += a[i][i];
        secondDiag += a[i][n - 1 - i];
    }
}

int main() {
    int n, a[MAX][MAX];
    cin >> n;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j) cin >> a[i][j];

    long long mainDiag, secondDiag;
    diagonalSums(a, n, mainDiag, secondDiag);
    cout << "Cheo chinh: " << mainDiag << '\n';
    cout << "Cheo phu: " << secondDiag;
    return 0;
}
```

Bài 25. Chuyển vị ma trận ★

1. Nêu bài toán

Cho ma trận A có m hàng và n cột. Hãy tạo ma trận chuyển vị B có n hàng và m cột, trong đó $B[j][i] = A[i][j]$.

2. Input

Dòng 1: m và n ($1 \leq m, n \leq 100$). Tiếp theo là m dòng dữ liệu của A .

3. Output

In ma trận chuyển vị gồm n dòng, mỗi dòng m phần tử.

4. Ví dụ

INPUT	OUTPUT
2 3	1 4
1 2 3	2 5
4 5 6	3 6

5. Thuật toán giải

Bước 1: Tạo ma trận B .

Bước 2: Duyệt mọi phần tử $A[i][j]$ và gán $B[j][i] = A[i][j]$.

Bước 3: Hàm `transpose()` thực hiện phép chuyển vị; `main()` in B theo kích thước $n \times m$.

6. Code minh họa

```
#include <iostream>
```

```
using namespace std;

const int MAX = 100;

void transpose(const int a[][MAX], int m, int n, int b[][MAX]) {
    for (int i = 0; i < m; ++i)
        for (int j = 0; j < n; ++j)
            b[j][i] = a[i][j];
}

int main() {
    int m, n, a[MAX][MAX], b[MAX][MAX];
    cin >> m >> n;
    for (int i = 0; i < m; ++i)
        for (int j = 0; j < n; ++j) cin >> a[i][j];

    transpose(a, m, n, b);
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) {
            if (j) cout << ' ';
            cout << b[i][j];
        }
        cout << '\n';
    }
    return 0;
}
```

Bài 26. Tìm giá trị lớn nhất và tất cả vị trí xuất hiện

1. Nêu bài toán

Cho ma trận $m \times n$ số nguyên. Hãy tìm giá trị lớn nhất và in tất cả các vị trí có giá trị bằng giá trị lớn nhất. Hàng và cột được đánh số từ 1.

2. Input

Dòng 1: m, n . Tiếp theo là m dòng dữ liệu.

3. Output

Dòng 1 in giá trị lớn nhất. Các dòng sau in cặp (hàng, cột) nơi giá trị lớn nhất xuất hiện.

4. Ví dụ

INPUT	OUTPUT
2 3	Max: 9
1 9 3	(1, 2)
9 5 9	(2, 1)
	(2, 3)

5. Thuật toán giải

Bước 1: Gán $\text{maxVal} = a[0][0]$, duyệt toàn ma trận để tìm giá trị lớn nhất.

Bước 2: Duyệt ma trận lần thứ hai.

Bước 3: Khi $a[i][j] == \text{maxVal}$, in vị trí $(i+1, j+1)$.

6. Code minh họa

```
#include <iostream>
using namespace std;

int main() {
    const int MAX = 100;
    int m, n, a[MAX][MAX];
    cin >> m >> n;
    for (int i = 0; i < m; ++i)
        for (int j = 0; j < n; ++j) cin >> a[i][j];

    int maxVal = a[0][0];
    for (int i = 0; i < m; ++i)
        for (int j = 0; j < n; ++j)
            if (a[i][j] > maxVal) maxVal = a[i][j];
}
```



```

cout << "Max: " << maxVal << '\n';
for (int i = 0; i < m; ++i)
    for (int j = 0; j < n; ++j)
        if (a[i][j] == maxVal)
            cout << '(' << i + 1 << ", " << j + 1 << ")\n";
return 0;
}

```

Bài 27. Đếm số nguyên tố trong ma trận

1. Nêu bài toán

Cho ma trận $m \times n$ số nguyên. Hãy đếm có bao nhiêu phần tử là số nguyên tố. Số nguyên tố là số nguyên lớn hơn 1 và chỉ có hai ước dương là 1 và chính nó.

2. Input

Dòng 1: m, n . Tiếp theo là m dòng dữ liệu ma trận.

3. Output

In số lượng phần tử là số nguyên tố.

4. Ví dụ

INPUT	OUTPUT
<pre> 3 3 1 2 3 4 5 6 7 8 9 </pre>	<pre> 4 </pre>

5. Thuật toán giải

Bước 1: Với mỗi phần tử x , nếu $x \leq 1$ thì không phải số nguyên tố.

Bước 2: Nếu $x > 1$, thử các ước d từ 2 đến khi $d \cdot d > x$; nếu x chia hết cho d thì x không nguyên tố.

Bước 3: Tăng biến đếm khi không tìm thấy ước nào.

6. Code minh họa

```

#include <iostream>
using namespace std;

int main() {
    const int MAX = 100;
    int m, n, a[MAX][MAX];
    cin >> m >> n;
    for (int i = 0; i < m; ++i)
        for (int j = 0; j < n; ++j) cin >> a[i][j];

    int count = 0;
    for (int i = 0; i < m; ++i) {
        for (int j = 0; j < n; ++j) {
            int x = a[i][j];
            bool prime = (x > 1);
            for (int d = 2; prime && d <= x / d; ++d)
                if (x % d == 0) prime = false;
            if (prime) ++count;
        }
    }
    cout << count;
    return 0;
}

```

Bài 28. Tìm hàng có tổng lớn nhất không dùng hàm

1. Nêu bài toán

Cho ma trận $m \times n$. Hãy tìm hàng có tổng các phần tử lớn nhất. Nếu nhiều hàng bằng nhau, chọn hàng có số thứ tự nhỏ nhất.

2. Input

Dòng 1: m, n. Tiếp theo là m dòng ma trận.

3. Output

In số thứ tự hàng có tổng lớn nhất và giá trị tổng đó.

4. Ví dụ

INPUT	OUTPUT
3 3 1 2 3 4 5 6 -1 0 1	Hang: 2 Tong: 15

5. Thuật toán giải

Bước 1: Duyệt từng hàng.

Bước 2: Với mỗi hàng, đặt sum = 0 và cộng tất cả phần tử của hàng.

Bước 3: Nếu sum lớn hơn bestSum hiện tại thì cập nhật bestSum và bestRow.

6. Code minh họa

```
#include <iostream>
using namespace std;

int main() {
    const int MAX = 100;
    int m, n, a[MAX][MAX];
    cin >> m >> n;
    for (int i = 0; i < m; ++i)
        for (int j = 0; j < n; ++j) cin >> a[i][j];

    int bestRow = 0;
    long long bestSum = 0;
    for (int j = 0; j < n; ++j) bestSum += a[0][j];

    for (int i = 1; i < m; ++i) {
        long long sum = 0;
        for (int j = 0; j < n; ++j) sum += a[i][j];
        if (sum > bestSum) {
            bestSum = sum;
            bestRow = i;
        }
    }
    cout << "Hang: " << bestRow + 1 << '\n';
    cout << "Tong: " << bestSum;
    return 0;
}
```

Bài 29. Nhân hai ma trận**1. Nêu bài toán**

Cho ma trận A kích thước m x n và ma trận B kích thước n x p. Hãy tính ma trận tích C = A x B kích thước m x p.

2. Input

Dòng 1: m, n, p (1 ≤ m,n,p ≤ 100). Tiếp theo m dòng của A, sau đó n dòng của B.

3. Output

In ma trận C gồm m dòng, mỗi dòng p số nguyên.

4. Ví dụ

INPUT	OUTPUT
2 3 2 1 2 3 4 5 6 7 8	58 64 139 154

9 10
11 12

5. Thuật toán giải

Bước 1: Khởi tạo từng phần tử $C[i][j] = 0$.

Bước 2: Với mỗi cặp (i, j) , duyệt k từ 0 đến $n-1$ và cộng $A[i][k] * B[k][j]$.

Bước 3: In C theo m hàng và p cột. Độ phức tạp $O(m*n*p)$.

6. Code minh họa

```
#include <iostream>
using namespace std;

int main() {
    const int MAX = 100;
    int m, n, p;
    long long a[MAX][MAX], b[MAX][MAX], c[MAX][MAX] = {};
    cin >> m >> n >> p;
    for (int i = 0; i < m; ++i)
        for (int j = 0; j < n; ++j) cin >> a[i][j];
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < p; ++j) cin >> b[i][j];

    for (int i = 0; i < m; ++i)
        for (int j = 0; j < p; ++j)
            for (int k = 0; k < n; ++k)
                c[i][j] += a[i][k] * b[k][j];

    for (int i = 0; i < m; ++i) {
        for (int j = 0; j < p; ++j) {
            if (j) cout << ' ';
            cout << c[i][j];
        }
        cout << '\n';
    }
    return 0;
}
```

Bài 30. Kiểm tra ma trận đối xứng

1. Nêu bài toán

Cho ma trận vuông $n \times n$. Ma trận được gọi là đối xứng nếu $a[i][j] = a[j][i]$ với mọi i, j . Hãy kiểm tra ma trận đã cho có đối xứng hay không.

2. Input

Dòng 1: n ($1 \leq n \leq 100$). Tiếp theo là n dòng, mỗi dòng n số nguyên.

3. Output

In YES nếu ma trận đối xứng, ngược lại in NO.

4. Ví dụ

INPUT	OUTPUT
3 1 2 3 2 4 5 3 5 6	YES

5. Thuật toán giải

Bước 1: Chỉ cần kiểm tra các phần tử phía trên đường chéo chính, tức $j > i$.

Bước 2: Nếu tồn tại $a[i][j] \neq a[j][i]$, kết luận không đối xứng và dừng kiểm tra.

Bước 3: Nếu không phát hiện cặp sai khác, in YES.

6. Code minh họa

```
#include <iostream>
```

```
using namespace std;

int main() {
    const int MAX = 100;
    int n, a[MAX][MAX];
    cin >> n;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j) cin >> a[i][j];

    bool symmetric = true;
    for (int i = 0; i < n && symmetric; ++i) {
        for (int j = i + 1; j < n; ++j) {
            if (a[i][j] != a[j][i]) {
                symmetric = false;
                break;
            }
        }
    }
    cout << (symmetric ? "YES" : "NO");
    return 0;
}
```

TÀI LIỆU THAM KHẢO

Bjarne Stroustrup. The C++ Programming Language, Fourth Edition. Addison-Wesley, 2013. Tham khảo chính: Chapter 7 - Pointers, Arrays, and References; Chapter 12 - Functions.

